

DP & RL: Assignment week 1

Ángel A. Gil (2788988)

MSc AI, VU - Group 150 - 10/11/2023

1 Introduction

In this report, we address a **stochastic variant of the knapsack problem** where items of unknown sizes are presented sequentially. The objective is to **maximize the reward** by selectively **taking items without exceeding the knapsack's capacity**.

2 Problem Formulation

We consider a finite horizon of $T = 10$ time steps, representing the sequential presentation of items. The size of each item is a random variable s_t with a discrete uniform distribution over the set $\{1, 2, \dots, 10\}$. The knapsack has a capacity of $W = 10$. At each time step t , where $1 \leq t \leq N$, an action a_t is taken to accept or reject the presented item based on its size and the remaining capacity. The state transition function $\Gamma_t(x, a)$ retains the current capacity if the item is not taken, or decreases it by the size of the item if taken (x corresponds to the current capacity). The objective is to determine the policy that maximizes the expected reward.

3 Optimal Policy Proof (*question a*)

Claim: *If accepting an item of size s at time t is optimal, accepting any item of size less than s is also optimal.*

Proof: If $\forall t \in T : V(x - s, t) \leq V(x - s', t)$ holds, the claim is true by definition. The base case holds, since $V(x, 0)$ is zero for all possible states. All that is left is proving this statement holds for $t + 1$ assuming that it is true until an arbitrary t .

For any $s' < s$, the capacity $x' = x - s'$ will be greater than $x - s$. Due to the non-decreasing nature of the value function $V_t(x)$ with respect to capacity (proven in Ex.8 tutorial 1), we have $V_{t+1}(x') \geq V_{t+1}(x - s)$, independently of t . Another way to see it: at least the same arrangement that is valid for x , will be valid for x' , since in the latter case we have the same amount of items (same reward) but we are left with more space available. It is not necessary to check the cases where $x - s < 0$ since it does not apply to the original claim and, thus, these cases can be deprecated.

4 Identification of $\mathcal{X}$ and $\mathcal{A}$ (*question b*)

Given the problem formulation and the optimal policy conditions established in the proof, we define our state space $\mathcal{X}$ as the set of all possible knapsack capacities from 0 to 10 at any given time. The action space consists on the binary action (to take the item or not) for each possible item size. This would lead to $\mathcal{A}$ being of size 2^{10} . Nevertheless, from the previous result, we can reduce this number drastically in practice. If we evaluate decreasing sizes, the first time that $V_{t+1}(x - s) + 1 > V_{t+1}(x)$ for a weight s , (i.e.) if taking

the item is expected to be better than not taking it, then the optimal policy for smaller sizes will be to also take the items, so the optimal sequences will be all 0s until they become all 1s.

5 Algorithm Implementation and Results (*question c*)

Lets review the DP algorithm implemented to construct a value function table V and a policy matrix policy. Initialized to zeros, V is filled by iterating backwards from the last item to the first, considering each remaining capacity w . For each state, we calculate the expected value of taking an item, (*expected_value_take*), versus not taking it, (*expected_value_not_take*). This decision is informed by a computed (*max_fit*) value, representing the largest item size that improves the expected reward.

The policy matrix is populated accordingly, where a value closer to 1 indicates a higher likelihood of taking the item, signifying its contribution to an optimal strategy. This is visualized in the heatmap 1, with darker areas suggesting a decision to take the item is preferred, and lighter areas indicating otherwise. **NOTE (on the policy):** I realised too late that the policy should indeed be 1 or 0, since it represents the action taken and therefore it does not make sense to have "likelihood" of taking items. I apologize for this error, I ran out of time and will not be able to fix it.

The expected maximal reward is obtained from $V[0][W]$, encapsulating the best outcome achievable under the optimal policy. In this case, it is ≈ 3.406 , which makes sense taking into consideration that the average weight would be $\sum \frac{n*(n+1)}{2*10} = 5.5$, where n goes from 1 to 10. This would result in slightly less than 2 items expected on average, but we probably avoid taking the heaviest items, which proves the usefulness of the optimal policy.

Efficiency is key in the implementation, with the algorithm running in polynomial time in practice, making it feasible for larger scale problems.

6 Simulation and Analysis (*question d*)

In order to evaluate the performance of the optimal policy derived from the dynamic programming approach, we simulated the decision-making process 1000 times. For each simulation run, the process unfolds over the given horizon of 10 items, with each item's size randomly drawn from a uniform distribution. The decision to accept or reject an item is governed by the optimal policy, which considers the current state, defined by the remaining capacity and the sequence of items thus far. When the policy advises accepting an item and it fits in the knapsack, the reward increases by one, and the knapsack's remaining capacity is adjusted accordingly.

This simulation process is not merely a passive replay of the optimal policy but an active re-enactment that accounts for the randomness of each item's size, mirroring the variability one would encounter in real-world scenarios. It's important to note that while the optimal policy remains static during the simulations, the outcomes vary due to the stochastic nature of the item sizes, thereby introducing a spectrum of possible reward outcomes. This variability is what gives rise to the distribution observed in the histogram 2. The x-axis denotes the total reward obtained in a single simulation, and the y-axis shows the frequency of each reward outcome. The shape of the distribution offers insights into the consistency and reliability of the optimal policy. Most simulations ended up in taking 3 to 4 items, while 2 and 5 are slightly less common occurrences and we have a few extreme cases where only 1 or up to 6 items were able to fit in the knapsack. These occur when the instantiation resulted in overall really big items or really small items, not allowing the policy to make a big difference.

The mean reward from the simulations, which was approximately 3.404 in this case, with relatively big standard deviation of ≈ 0.91 is really close to the expected maximal reward. This is what was expected to happen. Through running the simulation up to 10 more times, the values always fell close to 3.4

7 Appendix: Figures

7.1 Heatmap (question c)

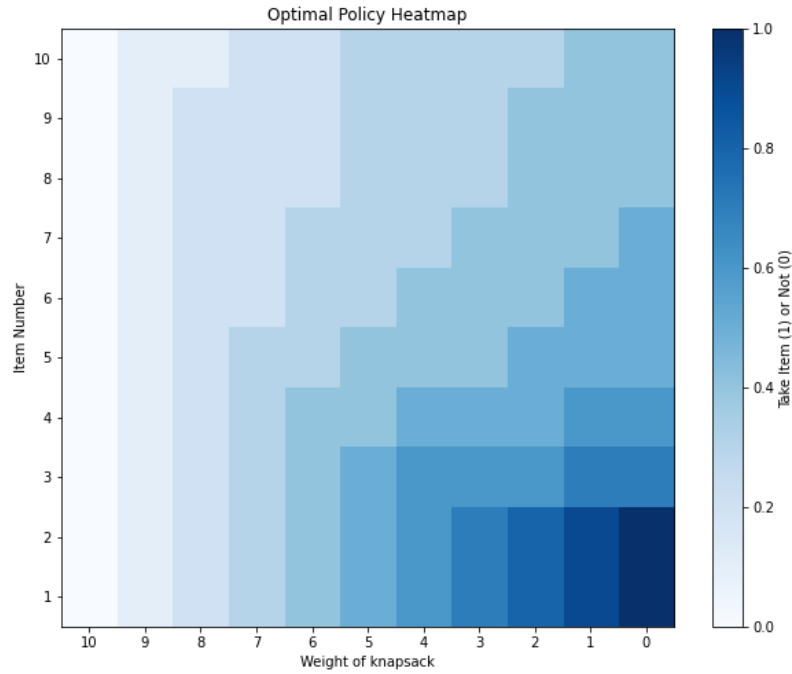


Figure 1: Heatmap for the optimal policy. A value closer to 1 (darker) indicates a higher likelihood of taking the item

7.2 Histogram (question d)

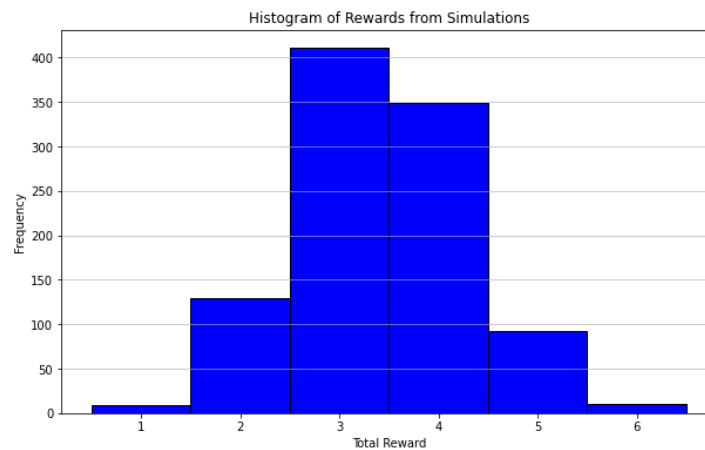


Figure 2: Histogram of the 1000 simulations. Most of the outcomes are either 3 or 4 items in the knapsack